

Wednesday 5:46 PM



lab_one.pdf
PDF

Here is my lab, run me though what we doing, what language, what is the task, how can i go about it



How do the files look from a glance, also why is there a
claude.md file?

Worked for 1m 6s >

Ari, the next coherent feature is **editing an existing task and changing its status**. The marker will specifically check that an edit survives a page reload, and statuses must remain limited to Todo, In-Progress and Complete. lab_one

Assuming archiving is working, commit it first:

```
git add .
git commit -m "Add task archiving and archived task view"
git push
```

1. Replace src/app/actions.ts

Use the complete file below so the three actions remain together:

```
"use server";

import { revalidatePath } from "next/cache";
import { TaskStatus } from "@generated/prisma/client";
import { prisma } from "@lib/prisma";

export async function createTask(formData: FormData) {
  const title = formData.get("title")?.toString().trim();
  const description = formData.get("description")?.toString().trim();
  const topic = formData.get("topic")?.toString().trim();
  const dueDateValue = formData.get("dueDate")?.toString();

  if (!title || !description || !topic || !dueDateValue) {
    throw new Error("All task fields are required.");
  }

  const dueDate = new Date(dueDateValue);

  if (Number.isNaN(dueDate.getTime())) {
    throw new Error("The due date is invalid.");
  }
}
```

```
    await prisma.task.create({
      data: {
        title,
        description,
        topic,
        dueDate,
      },
    });

    revalidatePath("/");
  }

export async function updateTask(formData: FormData) {
  const taskId = Number(formData.get("taskId"));
  const title = formData.get("title")?.toString().trim();
  const description = formData.get("description")?.toString().trim();
  const topic = formData.get("topic")?.toString().trim();
  const dueDateValue = formData.get("dueDate")?.toString();
  const statusValue = formData.get("status")?.toString();

  if (!Number.isInteger(taskId) || taskId <= 0) {
    throw new Error("Invalid task ID.");
  }

  if (!title || !description || !topic || !dueDateValue || !statusValue) {
    throw new Error("All task fields are required.");
  }

  if (
    !Object.values(TaskStatus).includes(statusValue as TaskStatus)
  ) {
    throw new Error("Invalid task status.");
  }

  const dueDate = new Date(dueDateValue);

  if (Number.isNaN(dueDate.getTime())) {
    throw new Error("The due date is invalid.");
  }

  await prisma.task.update({
    where: {
      id: taskId,
    },
    data: {
      title,
      description,
      topic,
      dueDate,
      status: statusValue as TaskStatus,
    },
  });
}
```

```

    revalidatePath("/");
  }

export async function archiveTask(formData: FormData) {
  const taskId = Number(formData.get("taskId"));

  if (!Number.isInteger(taskId) || taskId <= 0) {
    throw new Error("Invalid task ID.");
  }

  await prisma.task.update({
    where: {
      id: taskId,
    },
    data: {
      archivedAt: new Date(),
    },
  });

  revalidatePath("/");
}

```

2. Replace src/app/page.tsx

```

import {
  archiveTask,
  createTask,
  updateTask,
} from "../actions";
import { prisma } from "@lib/prisma";

function formatDateTimeInput(date: Date) {
  const timezoneOffset = date.getTimezoneOffset() * 60_000;

  return new Date(date.getTime() - timezoneOffset)
    .toISOString()
    .slice(0, 16);
}

function formatStatus(status: string) {
  if (status === "IN_PROGRESS") {
    return "In-Progress";
  }

  if (status === "COMPLETE") {
    return "Complete";
  }
}

```



```
    return "Todo";
  }

export default async function Home() {
  const [activeTasks, archivedTasks] = await Promise.all([
    prisma.task.findMany({
      where: {
        archivedAt: null,
      },
      orderBy: {
        createdAt: "desc",
      },
    }),

    prisma.task.findMany({
      where: {
        archivedAt: {
          not: null,
        },
      },
      orderBy: {
        archivedAt: "desc",
      },
    }),
  ]);

  return (
    <main>
      <h1>Todo Application</h1>

      <section>
        <h2>Create a task</h2>

        <form action={createTask}>
          <div>
            <label htmlFor="title">Title</label>
            <input
              id="title"
              name="title"
              type="text"
              required
            />
          </div>

          <div>
            <label htmlFor="description">Description</label>
            <textarea
              id="description"
              name="description"
              required
            />
          </div>
        </form>
      </section>
    </main>
  );
}
```

```
<div>
  <label htmlFor="dueDate">Due date</label>
  <input
    id="dueDate"
    name="dueDate"
    type="datetime-local"
    required
  />
</div>

<div>
  <label htmlFor="topic">Topic</label>
  <input
    id="topic"
    name="topic"
    type="text"
    required
  />
</div>

  <button type="submit">Create task</button>
</form>
</section>

<section>
  <h2>Active tasks</h2>

  {activeTasks.length === 0 ? (
    <p>No active tasks.</p>
  ) : (
    <ul>
      {activeTasks.map((task) => (
        <li key={task.id}>
          <h3>{task.title}</h3>
          <p>{task.description}</p>
          <p>Topic: {task.topic}</p>
          <p>Status: {formatStatus(task.status)}</p>
          <p>Due: {task.dueDate.toLocaleString()}</p>

          <details>
            <summary>Edit task</summary>

            <form action={updateTask}>
              <input
                type="hidden"
                name="taskId"
                value={task.id}
              />

              <div>
                <label htmlFor={`title-${task.id}`}>
```

```
        Title
      </label>
      <input
        id={`title-${task.id}`}
        name="title"
        type="text"
        defaultValue={task.title}
        required
      />
    </div>

    <div>
      <label htmlFor={`description-${task.id}`}>
        Description
      </label>
      <textarea
        id={`description-${task.id}`}
        name="description"
        defaultValue={task.description}
        required
      />
    </div>

    <div>
      <label htmlFor={`dueDate-${task.id}`}>
        Due date
      </label>
      <input
        id={`dueDate-${task.id}`}
        name="dueDate"
        type="datetime-local"
        defaultValue={formatDateTimeInput(
          task.dueDate,
        )}
        required
      />
    </div>

    <div>
      <label htmlFor={`topic-${task.id}`}>
        Topic
      </label>
      <input
        id={`topic-${task.id}`}
        name="topic"
        type="text"
        defaultValue={task.topic}
        required
      />
    </div>

    <div>
```

```

<label htmlFor={`status-${task.id}`}>
  Status
</label>
<select
  id={`status-${task.id}`}
  name="status"
  defaultValue={task.status}
  required
>
  <option value="TODO">Todo</option>
  <option value="IN_PROGRESS">
    In-Progress
  </option>
  <option value="COMPLETE">
    Complete
  </option>
</select>
</div>

  <button type="submit">Save changes</button>
</form>
</details>

<form action={archiveTask}>
  <input
    type="hidden"
    name="taskId"
    value={task.id}
  />
  <button type="submit">Archive task</button>
</form>
</li>
  )}}
</ul>
)}
</section>

<section>
  <h2>Archived tasks</h2>

  {archivedTasks.length === 0 ? (
    <p>No archived tasks.</p>
  ) : (
    <ul>
      {archivedTasks.map((task) => (
        <li key={task.id}>
          <h3>{task.title}</h3>
          <p>{task.description}</p>
          <p>Topic: {task.topic}</p>
          <p>Status: {formatStatus(task.status)}</p>
          <p>Due: {task.dueDate.toLocaleString()}</p>
          <p>

```

```

        Archived:{" "}
        {task.archivedAt?.toLocaleString()}
      </p>
    </li>
  )}
</ul>
)}
</section>
</main>
);
}

```

3. Test it

Run:

PowerShell

For one active task:

1. Open **Edit task**.
2. Change its title.
3. Change its description.
4. Change its topic.
5. Change its due date.
6. Change status to `In-Progress`.
7. Click **Save changes**.
8. Refresh the page.
9. Restart the server.

Everything should remain changed after both the reload and restart.

4. Commit it

Once editing works:

```

git add .
git commit -m "Add persistent task editing and status updates"
git push

```

This should be your fifth coherent commit. The next feature will be sorting by topic, status and due date.

lets do the next part

Worked for 48s >

Ari, next is **sorting the active task list by topic, status, or due date**. The lab explicitly checks all three during the walkthrough. lab_one

We'll use a URL parameter such as:

```
http://localhost:3000/?sort=topic
```

Next.js pages receive these values through the asynchronous `searchParams` property, and Prisma's `orderBy` option sorts the database results. Next.js +1

1. Update the imports in `src/app/page.tsx`

Add `Prisma` to your imports:

```
import { Prisma } from "@generated/prisma/client";
import { prisma } from "@lib/prisma";
import {
  archiveTask,
  createTask,
  updateTask,
} from "../actions";
```

2. Add the sorting types and functions

Place these functions above your `Home` component, underneath `formatStatus`:

```
type SortOption = "dueDate" | "topic" | "status";

type HomeProps = {
  searchParams: Promise<{
    sort?: string | string[];
  }>;
};

function isSortOption(value: string | undefined): value is SortOption {
  return (
```

```

    value === "dueDate" ||
    value === "topic" ||
    value === "status"
  );
}

function getTaskOrder(
  sort: SortOption,
): Prisma.TaskOrderByWithRelationInput[] {
  switch (sort) {
    case "topic":
      return [
        { topic: "asc" },
        { dueDate: "asc" },
      ];

    case "status":
      return [
        { status: "desc" },
        { dueDate: "asc" },
      ];

    case "dueDate":
    default:
      return [
        { dueDate: "asc" },
      ];
  }
}

```

The secondary due-date sorting keeps tasks with the same topic or status in a predictable order.

3. Update the start of the Home component

Replace:

TypeScript

with:

```

export default async function Home({
  searchParams,
}: HomeProps) {
  const parameters = await searchParams;

```

```
const requestedSort = Array.isArray(parameters.sort)
  ? parameters.sort[0]
  : parameters.sort;

const sort: SortOption = isSortOption(requestedSort)
  ? requestedSort
  : "dueDate";

const orderBy = getTaskOrder(sort);
```

Your database queries should come immediately afterwards.

4. Update both Prisma queries

Change your active query from:

```
prisma.task.findMany({
  where: {
    archivedAt: null,
  },
  orderBy: {
    createdAt: "desc",
  },
});
```

to:

```
prisma.task.findMany({
  where: {
    archivedAt: null,
  },
  orderBy,
});
```

Change your archived query to:

```
prisma.task.findMany({
  where: {
    archivedAt: {
      not: null,
    },
  },
  orderBy: {
    archivedAt: "desc",
  },
});
```



```
  },
  })
```

We are only making the **active list** sortable. The archived list remains sorted by the date it was archived.

5. Add the sorting form

Inside the Active Tasks section, immediately after:

TypeScript

add:

```
<form method="get">
  <label htmlFor="sort">Sort tasks by</label>

  <select
    id="sort"
    name="sort"
    defaultValue={sort}
  >
    <option value="dueDate">Due date</option>
    <option value="topic">Topic</option>
    <option value="status">Status</option>
  </select>

  <button type="submit">Apply sort</button>
</form>
```

Submitting a normal GET form places the selected value in the URL as a search parameter and causes the page to render using the new sorting option.  Next.js

6. Check the complete query section

The relevant part near the beginning of `Home` should now resemble:

```
export default async function Home({
  searchParams,
}: HomeProps) {
  const parameters = await searchParams;

  const requestedSort = Array.isArray(parameters.sort)
    ? parameters.sort[0]
```

```

      : parameters.sort;

const sort: SortOption = isSortOption(requestedSort)
  ? requestedSort
  : "dueDate";

const orderBy = getTaskOrder(sort);

const [activeTasks, archivedTasks] = await Promise.all([
  prisma.task.findMany({
    where: {
      archivedAt: null,
    },
    orderBy,
  }),
  prisma.task.findMany({
    where: {
      archivedAt: {
        not: null,
      },
    },
    orderBy: {
      archivedAt: "desc",
    },
  }),
]);

return (

```

7. Test all three options

Create several active tasks with:

- Different topics.
- Different statuses.
- Different due dates.

Then test:

Due date
Topic
Status

Confirm that:

- The order changes correctly.
- The selected option remains visible.
- Refreshing the page preserves the selected sorting because it remains in the URL.
- Creating, editing and archiving still work.

Also run:

PowerShell

Then commit:

```
git add .  
git commit -m "Add task sorting by topic status and due date"  
git push
```

This should be commit 6, but do not stop at six—we still need overdue highlighting, automated tests and documentation. Six is the floor, not a finish line.

    Sources

Branched from [Branch · Full-stack Todo App](#)

```
export default async function Home() {  
  const [activeTasks, archivedTasks] = await Promise.all([  
    prisma.task.findMany({  
      where: {  
        archivedAt: null,  
      },  
      orderBy: {  
        createdAt: "desc",  
      },  
    }],  
  ]),
```

Show more ▾

Ari, replace the beginning of your current `Home` function with the updated version. Do not place it inside the existing function—you are changing the function declaration and the active-task query.

1. Add this import at the top of `page.tsx`

TypeScript

Keep your existing imports as well.

2. Add these helpers above `Home`

Place them after `formatStatus()` but before:

TypeScript

```
type SortOption = "dueDate" | "topic" | "status";

type HomeProps = {
  searchParams: Promise<{
    sort?: string | string[];
  }>;
};

function isSortOption(
  value: string | undefined,
): value is SortOption {
  return (
    value === "dueDate" ||
    value === "topic" ||
    value === "status"
  );
}

function getTaskOrder(
  sort: SortOption,
): Prisma.TaskOrderByWithRelationInput[] {
  switch (sort) {
    case "topic":
      return [
        { topic: "asc" },
        { dueDate: "asc" },
      ];

    case "status":
      return [
        { status: "asc" },
        { dueDate: "asc" },
      ];

    case "dueDate":
    default:
```

```

    return [{ dueDate: "asc" }]];
  }
}

```

3. Replace your current function beginning

Replace this entire section:

```

export default async function Home() {
  const [activeTasks, archivedTasks] = await Promise.all([
    prisma.task.findMany({
      where: {
        archivedAt: null,
      },
      orderBy: {
        createdAt: "desc",
      },
    }),
    prisma.task.findMany({
      where: {
        archivedAt: {
          not: null,
        },
      },
      orderBy: {
        archivedAt: "desc",
      },
    }),
  ]);
}

```

with:

```

export default async function Home({
  searchParams,
}: HomeProps) {
  const parameters = await searchParams;

  const requestedSort = Array.isArray(parameters.sort)
    ? parameters.sort[0]
    : parameters.sort;

  const sort: SortOption = isSortOption(requestedSort)
    ? requestedSort
    : "dueDate";

  const orderBy = getTaskOrder(sort);
}

```

Your existing:

continues directly underneath that code.

Find this inside your returned JSX:

Immediately below the heading, add:

54/55

```
<option value="dueDate">Due date</option>
<option value="topic">Topic</option>
<option value="status">Status</option>
</select>

<button type="submit">Apply sort</button>
</form>
```

The structure should therefore be:

TypeScript

```
<section>
  <h2>Active tasks</h2>

  <form method="get">
    { /* sorting fields */ }
  </form>

  {activeTasks.length === 0 ? (
    // existing task-list code
  ) : (
    // existing task-list code
  )}
</section>
```

Only the **active task query** uses the selected sorting method. Archived tasks remain sorted by the time they were archived. This covers the required topic, status and due-date sorting in the marking walkthrough.

lab_one

  ...  Sources

ChatGPT can make mistakes. Check important info.